Security overview for Streamlit in Snowflake

This topic provides a security overview for system administrators managing Streamlit in Snowflake in their Snowflake accounts. Understanding the security model and implementing proper controls ensures that developers can build secure applications while administrators maintain governance over sensitive data and resources.

Security model

Streamlit in Snowflake follows Snowflake's comprehensive security model, which includes authentication, role-based access control, network policies, and data governance features. Apps are first-class Snowflake objects that integrate with existing security infrastructure.

Owner's rights execution

By default, Streamlit apps run with owner's rights, similar to stored procedures. This has the following consequences:

- Apps execute queries using the privileges of the app owner, not the viewer.
- The app owner's role determines what data and operations the app can access.
- Viewers can interact with the app without needing direct access to underlying tables or views.

This model eliminates the need for service account tokens and integrates seamlessly with Snowflake's authentication and ac
Understanding owner's rights and

As an alternative, you can configu
(Preview), which allows the app to
more information, see Restricted c

SNOWFLAKE'S USE OF COOKIES

We use cookies to enhance your experience, to display customized content, and to analyze site traffic. By clicking "Accept Cookies" or closing this banner, you provide your consent to our use of cookies. To learn more about the cookies we use and to set your own preferences, see our Cookie Statement.

ACCEPT ALL COOKIES REJECT ALL Cookies Settings

Shared responsibility r

Security responsibility is shared between Snowflake, account administrators, and app developers:

- Snowflake provides the secure platform, authentication, encryption, and security features.
- Administrators configure account-level security policies, manage roles and privileges, and audit app usage.
- App developers write secure code, handle secrets properly, and follow security best practices.

For more information about Snowflake's security model, see Snowflake's Shared Responsibility Model.

Content Security Policy

All Streamlit apps run within a Content Security Policy (CSP) that restricts which resources can be loaded. This policy provides defense-in-depth protection against cross-site scripting (XSS) and other code injection attacks. The CSP is not configurable at this time.

The CSP blocks the following external resources:

- Loading code (scripts, styles, fonts) from external domains
- Embedding apps in iframes from external domains

The CSP allows the following external resources:

- Images and media from HTTPS sources: Apps can load images and media files from any HTTPS URL, including external image hosting services and APIs that return images. This doesn't require an external access integration.
- Data URIs and blob URLs: Apps can use embedded data (data URIs) and dynamically generated content (blob URLs) for images and media. This supports features like displaying charts, diagrams, or user-uplo
- Mapbox and Carto resources: permitted to support mapping

Note

- For warehouse runtimes w the Anaconda terms to use packages from Anaconda.

SNOWFLAKE'S USE OF COOKIES

We use cookies to enhance your experience, to display customized content, and to analyze site traffic. By clicking "Accept Cookies" or closing this banner, you provide your consent to our use of cookies. To learn more about the cookies we use and to set your own preferences, see our Cookie Statement.

- Loading images or media from external domains is supported in Streamlit in Snowflake, but not in Snowflake Native App Framework.
- The CSP also blocks front-end calls that are generally considered unsafe, such as `eval()`.

This restrictive policy means that most third-party JavaScript libraries and custom components that rely on external scripts won't work in Streamlit apps. For more information about CSP limitations, see Loading external resources.

Essential security setup

The following security configurations are essential for a secure and well-functioning Streamlit in Snowflake environment.

Network access configuration

Configure network access to ensure that apps can communicate with Snowflake.

For all deployments:

- Add `*.snowflake.app` to your network allowlist to enable communication between Streamlit apps and Snowflake.
- For Streamlit apps using container runtimes, also add `*.snowflakecomputing.app` to your network allowlist.
- Ensure WebSockets are not blocked in your network configuration.

For more information, see You can't load the Streamlit app.

For private connectivity:

If your organization requires privat
or Google Cloud Private Service C
For more information, see Private

SNOWFLAKE'S USE OF COOKIES

We use cookies to enhance your experience, to display customized content, and to analyze site traffic. By clicking "Accept Cookies" or closing this banner, you provide your consent to our use of cookies. To learn more about the cookies we use and to set your own preferences, see our **Cookie Statement.**

Role-based access co

Establish a role hierarchy for mana

Recommended role structure:

- Creator roles: Roles with CREATE STREAMLIT privileges on schemas where apps will be deployed.
- Viewer roles: Roles with USAGE privileges on apps for end users.

The following example shows how to create a role hierarchy for Streamlit apps:

```
-- Create dedicated roles for Streamlit
CREATE ROLE streamlit_developer;
CREATE ROLE streamlit_viewer;

-- Grant hierarchy
GRANT ROLE streamlit_viewer TO ROLE streamlit_developer;

-- Grant privileges for app creation
GRANT USAGE ON DATABASE streamlit_db TO ROLE streamlit_developer;
GRANT USAGE ON SCHEMA streamlit_db.apps TO ROLE streamlit_developer;
GRANT CREATE STREAMLIT ON SCHEMA streamlit_db.apps TO ROLE streamlit_developer;
GRANT USAGE ON COMPUTE_POOL streamlit_compute_pool TO ROLE streamlit_developer;
GRANT USAGE ON INTEGRATION python_package_index TO ROLE streamlit_developer;
```

The app developer also needs USAGE on `streamlit_wh`, but this is inherited from the viewer role. For more information about required privileges, see Privileges required to create and use a Streamlit app.

Secrets management

Configure proper secrets management for apps that access external services or sensitive credentials:

1. Enable secrets access for apps by granting appropriate privileges:

```
-- Grant privileges on
GRANT READ ON SECRET m
GRANT USAGE ON INTEGRA
```

2. For container runtime apps, cr
embedding secrets in app coc

For more information, see Manage

SNOWFLAKE'S USE OF COOKIES

We use cookies to enhance your experience, to display customized content, and to analyze site traffic. By clicking "Accept Cookies" or closing this banner, you provide your consent to our use of cookies. To learn more about the cookies we use and to set your own preferences, see our Cookie Statement.

Context functions and row-level security

In warehouse runtimes, if your apps use context functions (such as `CURRENT_USER()`) or access tables with row access policies, grant the global READ SESSION privilege to app owner roles:

```
USE ROLE ACCOUNTADMIN;
GRANT READ SESSION ON ACCOUNT TO ROLE streamlit_developer;
```

Note

Warehouse-runtime apps using `CURRENT_ROLE()` in row access policies will always return the app owner's role, not the viewer's role, because apps run with owner's rights by default.

For more information and examples, see Row access policies in Streamlit in Snowflake.

In container runtimes, context functions on owner's rights connections will return values from the owner role's context and so are not appropriate for user-targeted row access policies. However, restricted caller's rights connections return the viewer's context. For more information, see Restricted caller's rights and Streamlit in Snowflake.

Container runtimes only: Package repository access and security

FEATURE — GENERALLY AVAILABLE

Not supported in government regions.

Configure one or more package indexes for container runtimes.

Container runtimes can install pac
package sources using managed p
default package index, PyPI. Rega
external access integration (EAI) t

Using a managed package index p

- This helps prevent supply cha
- It allows you to control which

SNOWFLAKE'S USE OF COOKIES

We use cookies to enhance your experience, to display customized content, and to analyze site traffic. By clicking "Accept Cookies" or closing this banner, you provide your consent to our use of cookies. To learn more about the cookies we use and to set your own preferences, see our Cookie Statement.

- It provides audit trails for package installations.

For more information about how developers use EAIs to manage dependencies, see Managing dependencies for container runtimes. For more information about setting up a managed package repository with authentication, see Example: Authenticate to a private JFrog Artifactory repository.

Set up a PyPI EAI for app developers

Container-runtime apps attempt to install dependencies from PyPI by default. Snowflake provides a managed network rule, `SNOWFLAKE.EXTERNAL_ACCESS.PYPI_RULE`, that allows egress to PyPI. You can use this rule to create a PyPI EAI without defining your own network rule. For more information about managed network rules, see Snowflake-managed egress network rules.

The following SQL commands create a PyPI EAI using the Snowflake-managed network rule and grant USAGE to an app-development role:

```
USE ROLE ACCOUNTADMIN;

CREATE OR REPLACE EXTERNAL ACCESS INTEGRATION pypi_access_integration
  ALLOWED_NETWORK_RULES = (snowflake.external_access.pypi_rule)
  ENABLED = true;

GRANT USAGE ON INTEGRATION pypi_access_integration TO ROLE app_developer_role;
```

Warehouse runtimes only: External offerings terms

Warehouse runtimes use conda to manage your app's dependencies. If you want to use Mapbox in your apps, you must acknowledge the External Offerings Terms.

For information about using this package, see Using third-party packages from Anaconda.

SNOWFLAKE'S USE OF COOKIES

We use cookies to enhance your experience, to display customized content, and to analyze site traffic. By clicking "Accept Cookies" or closing this banner, you provide your consent to our use of cookies. To learn more about the cookies we use and to set your own preferences, see our **Cookie Statement.**

Available security

The following security features are

External access integr

Control which external networks and services your apps can access:

- Create network rules to define allowed endpoints, including package indexes.
- Create external access integrations that reference network rules and authentication secrets.
- Assign external access integrations to Streamlit apps.

This prevents apps from making unauthorized outbound connections and provides audit trails for external access.

For more information, see External network access in Streamlit in Snowflake.

Git integration

Integrate Streamlit apps with Git repositories for version control and change tracking:

- Grant appropriate privileges on Git repository objects (READ, WRITE, or OWNERSHIP).
- Use Git integration to maintain audit trails of code changes.
- Implement code review processes before deploying changes to production apps.

For more information, see Sync Streamlit in Snowflake apps with a Git repository.

Private connectivity

For organizations with strict network security requirements, configure private connectivity to ensure all Streamlit traffic remains within your private network. Streamlit in Snowflake supports the following private connectivity options:

- AWS PrivateLink
- Azure Private Link
- Google Cloud Private Service Connect

Private connectivity eliminates exp
isolation.

For more information, see Private

SNOWFLAKE'S USE OF COOKIES

We use cookies to enhance your experience, to display customized content, and to analyze site traffic. By clicking "Accept Cookies" or closing this banner, you provide your consent to our use of cookies. To learn more about the cookies we use and to set your own preferences, see our **Cookie Statement.**

Logging and tracing

Enable logging to monitor app beh

- Configure an event table for your account. For more information, see Event table overview.
- For warehouse runtimes, set appropriate log and trace levels for databases containing Streamlit apps. For more information, see Setting levels for logging, metrics, and tracing.
- Review logs regularly for security events, errors, and unusual behavior.

For container runtimes, Snowflake automatically captures standard output and standard error from the container and stores them in the account's event table. No additional configuration is needed.

For more information, see Logging and tracing for Streamlit in Snowflake.

Limit a user's access to only Streamlit in Snowflake

To restrict a user to only access Streamlit in Snowflake and prevent them from accessing other parts of Snowflake, an account administrator can add a custom user property via SQL or SCIM attribute.

- To restrict a user, use the ALTER USER SQL command to set the ALLOWED_INTERFACES property to include STREAMLIT:

```
ALTER USER <user_name> SET ALLOWED_INTERFACES = (STREAMLIT);
```

If you're provisioning users with SCIM APIs, you can set the same setting using the custom attribute `allowedInterfaces`. For more information about SCIM custom attributes, see SCIM user API reference.

After Streamlit-only access is configured, the user can't access any part of Snowflake except the Streamlit in Snowflake apps for which they have permission. Additionally, they can only access the app-viewer URL for those apps. If a Streamlit-only user attempts to navigate anywhere in Snowflake, including

Redirect app viewers t

An account administrator can cont
(IdP) when an unauthenticated vie
user's login flow.

SNOWFLAKE'S USE OF COOKIES

We use cookies to enhance your experience, to display customized content, and to analyze site traffic. By clicking "Accept Cookies" or closing this banner, you provide your consent to our use of cookies. To learn more about the cookies we use and to set your own preferences, see our **Cookie Statement.**

- To redirect unauthenticated users from app-viewer URLs to your IdP, use the ALTER ACCOUNT SQL command to set the LOGIN_IDP_REDIRECT account property to include STREAMLIT:

```
ALTER ACCOUNT SET LOGIN_IDP_REDIRECT = (STREAMLIT = <your_security_integrati
```

For more information about configuring your Snowflake account to use an IdP, see the following topics:

- Configuring Snowflake to use federated authentication.
- Configuring an identity provider (IdP) for Snowflake.

Best practices for administrators

The following best practices help maintain a secure Streamlit environment.

Use dedicated roles and schemas:

- Create separate schemas for development, testing, and production apps.
- Use different roles for each environment to prevent accidental changes to production apps.
- Grant production app ownership to service roles rather than individual user accounts.

Implement least privilege access:

- Grant only the minimum required privileges to each role.
- Regularly review and audit role memberships and privileges.
- Avoid granting ACCOUNTADMIN or other powerful roles to app owner roles unless absolutely necessary.

Manage app lifecycle:

- Establish processes for app a
- Require code reviews before p
- Document which apps access
- Regularly review and remove

SNOWFLAKE'S USE OF COOKIES

We use cookies to enhance your experience, to display customized content, and to analyze site traffic. By clicking "Accept Cookies" or closing this banner, you provide your consent to our use of cookies. To learn more about the cookies we use and to set your own preferences, see our Cookie Statement.

Monitor resource usage:

- Set appropriate warehouse sizes for app workloads.
- Monitor compute costs and set up alerts for unusual usage patterns.
- For container runtimes, configure compute pools with appropriate MIN_NODES and MAX_NODES settings.
- Use separate warehouses for different app environments to isolate costs and resources.

For more information about resource management, see Managing costs for Streamlit in Snowflake and Runtime environments for Streamlit apps.

Use secure app development practices:

- Never embed credentials or API keys directly in app code.
- Use Snowflake secrets for storing sensitive information.
- Validate and sanitize user inputs to prevent SQL injection.
- Limit the data exposed through apps to only what viewers need to see.
- Test apps thoroughly before sharing with wider audiences.

For more information about owner's rights security considerations, see Owner's rights and app security.

Perform regular security audits:

- Review which roles have CREATE STREAMLIT privileges.
- Audit which apps access which data sources.
- Review external access integrations and network rules.
- Check for apps owned by former employees or inactive accounts.
- Review Git repository access and commit history.

Use the following queries to audit your Streamlit apps:

```
-- List all Streamlit apps
SHOW STREAMLITS;

-- Check privileges on a s
SHOW GRANTS ON STREAMLIT s

-- List all roles with CRE
SHOW GRANTS OF CREATE STRE
```


SNOWFLAKE'S USE OF COOKIES

We use cookies to enhance your experience, to display customized content, and to analyze site traffic. By clicking "Accept Cookies" or closing this banner, you provide your consent to our use of cookies. To learn more about the cookies we use and to set your own preferences, see our **Cookie Statement.**